Logo VUT

Technical Report: Advanced Off-Policy Methods in Deep Reinforcement Learning

Author: Eduardo Martín Postigo **Subject:** Comparative Analysis of DQN, TD3, and SAC

1. Introduction

Modern Deep Reinforcement Learning (DRL) has evolved from simple tabular methods to complex architectures capable of handling high-dimensional state spaces and continuous action domains. This report examines the transition from value-based discrete methods to state-of-the-art actor-critic stochastic frameworks, focusing on the mechanical improvements that provide stability and efficiency.

---

2. Core Definitions

To classify any Reinforcement Learning algorithm, we evaluate it across these four fundamental dimensions.

A. Action Spaces

- **Discrete:** The agent chooses from a finite, fixed set of actions.
  - *LaTeX:* $a \in 0, 1, \ldots, n-1$.
  - *Analogy:* Using a D-pad or a keyboard (binary choices).
- **Continuous:** The agent outputs real-valued vectors, allowing for infinite precision within bounds.
  - *LaTeX:* $a \in \mathbb{R}^d$ typically within $[-1, 1]$.
  - *Analogy:* Using a steering wheel or a throttle pedal (gradient choices).

B. Policy Types (The Decision Logic)

- **Deterministic:** Maps a state directly to a single, specific action.
  - *Formula:* $a = \mu(s)$.
  - *Behavior:* Exploitative and precise; requires external noise (like Gaussian) to explore.
- **Stochastic:** Maps a state to a probability distribution (usually a Gaussian mean μ and variance σ).
  - *Formula:* $a \sim \pi(a|s)$.
  - *Behavior:* Naturally exploratory; better for complex environments and handling uncertainty.

C. Paradigms (The Data Efficiency)

- **On-Policy:** The agent learns only from data collected by its *current* version. Data is discarded after the update.
  - *Example:* **PPO**.
  - *Trait:* Stable but "expensive" (requires constant new environment interactions).
- **Off-Policy:** The agent learns from a **Replay Buffer** containing experiences from past versions of itself.

- *Example:* **DQN, SAC, TD3**.
- *Trait:* Highly sample-efficient; reuses data many times to squeeze out more learning.

D. Architectures (The Brain Structure)

- **Value-Based:** Focuses exclusively on estimating the "quality" (Q) of every possible action. The policy is implicitly to "pick the highest Q."
  - *Constraint:* Hard to use in continuous spaces because you cannot calculate $\max$ over infinite values.
  - *Example:* **DQN**.
- **Actor-Critic:** Splits the model into two specialized networks.
  - **The Actor:** Learns *how* to behave (the policy).
  - **The Critic:** Learns to estimate the value of the Actor's actions.
  - *Example:* **DDPG, TD3, SAC, PPO**.

---

2. Comparative Methodology Overview

Once these few aspects are clarified, we can better understand the differences between each approach.

	DQN	**DDPG**	**TD3**	**SAC**	**PPO**
Paradigm	Off-Policy	Off-Policy	Off-Policy	Off-Policy	On-Policy
Action Space	Discrete	Continuous	Continuous	Continuous	Continuous
Policy Type	Deterministic	Deterministic	Deterministic	**Stochastic**	Stochastic
Architecture	Value-Based	Actor-Critic	Actor-Critic	Actor-Critic	Actor-Critic
Exploration	ϵ-greedy	Added Noise	Added Noise	**Max Entropy**	Probability Dist.
Stability	Medium	Low	High	**Very High**	High
Sample Efficiency	High	High	High	**Very High**	Low

---

3. DQN: Deep Q-Networks (The Value-Based Foundation)

DQN represents the first major successful integration of Deep Learning with Reinforcement Learning. It was the breakthrough that allowed agents to move beyond simple grids and solve tasks with high-dimensional sensory inputs, such as raw pixels, by approximating the optimal action-value function $Q^*(s, a)$ using a neural network.

3.1 Theoretical Framework

DQN is fundamentally a **Value-Based** method. It does not learn a policy directly; instead, it learns to estimate the "quality" of taking a specific action in a specific state.

The Bellman Equation

The agent improves its estimation by iteratively solving the Bellman Equation. The goal is to minimize the difference between the current Q-prediction and the "Target" (the immediate reward plus the discounted value of the next best state):

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

The Loss Function

To train the neural network (CNN), we minimize the **Mean Squared Error (MSE)** between our current prediction and the stable target:

$$L(\theta) = \mathbb{E} \left[(\underbrace{r + \gamma \max_{a'} Q(s', a'; \theta^{-})}{\text{Stable Target}} - \underbrace{Q(s, a; \theta)}{\text{Current Prediction}})^2 \right]$$

- γ **(Gamma):** The discount factor (usually 0.99).
- θ**:** Weights of the Policy Network.
- θ^{-}**:** Weights of the Target Network.

3.2 Evolution: From Q-Table to Neural Network

The core shift in DQN is the **representation of knowledge**.

- **Q-Learning (The Table):** Traditional RL uses a literal matrix (Q-Table). In a 96x96 pixel environment like `CarRacing-v2`, there are too many possible pixel combinations to store in a table.
- **DQN (The Approximator):** Uses a **Convolutional Neural Network (CNN)** as a function approximator. The network takes the pixel stack as input and predicts the Q-values for all available actions simultaneously. This allows the agent to **generalize**—it learns that a "red curb" means a turn, even if it has never seen that exact pixel arrangement before.

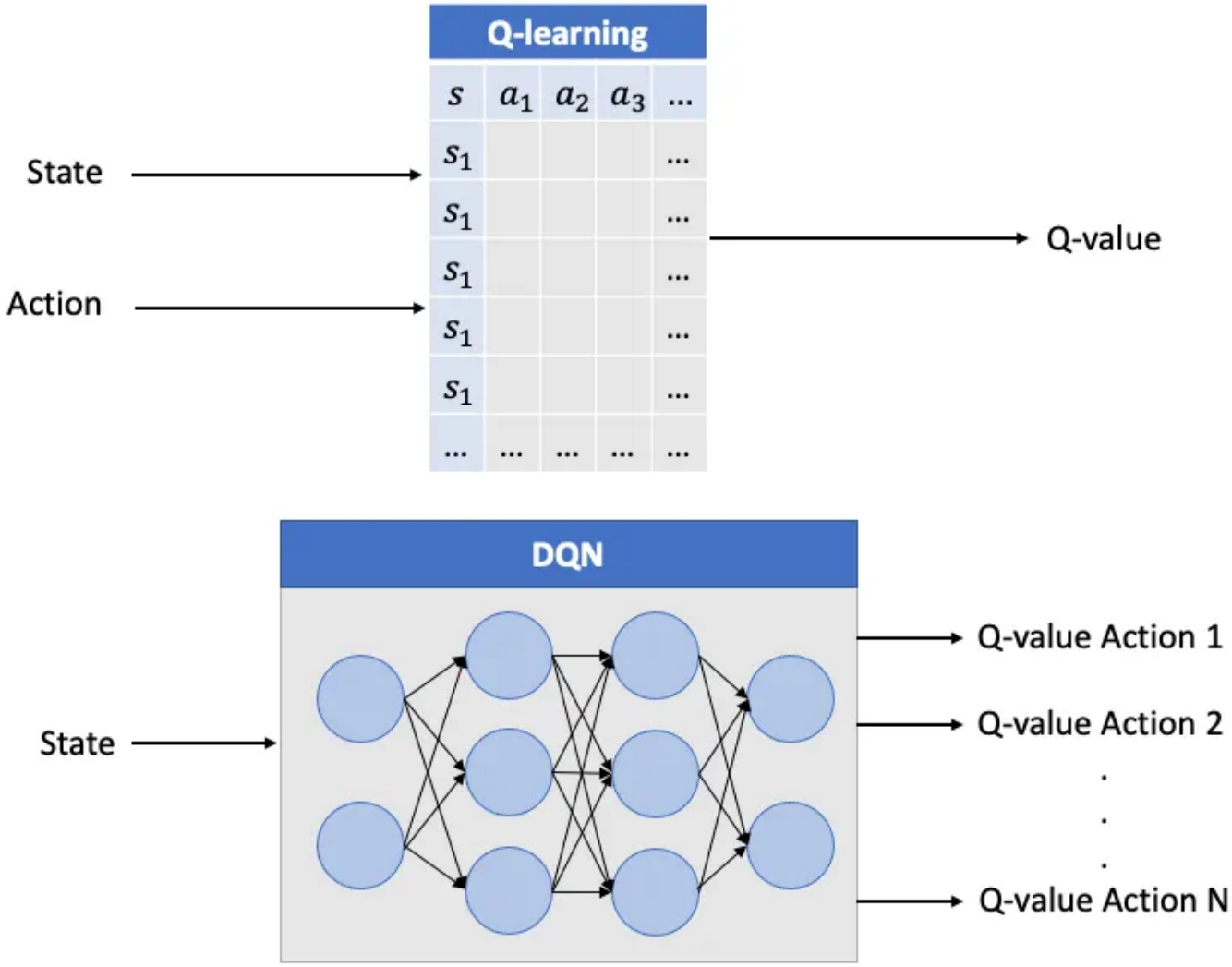

Figure 1: Structural comparison between traditional Tabular Q-Learning and Deep Q-Networks (DQN). Traditional Q-Learning relies on an exhaustive lookup table, which becomes computationally infeasible for high-dimensional sensory inputs. In contrast, DQN utilizes a neural network (CNN) to approximate Q-values, enabling the agent to generalize patterns and features from raw pixel observations. **Source:** Marta Comes Hernandez (Medium)

3.3 The Three Pillars of Stability

Standard neural networks are notoriously unstable when used for RL because the data is non-stationary (the agent's behavior changes as it learns). DQN solves this with three key mechanisms:

A. Epsilon-Greedy Strategy (Exploration vs. Exploitation)

Since DQN is a deterministic value-based method, the agent would always take the same "greedy" action if not forced to explore.

- **Mechanism:** With probability ϵ, the agent chooses a random action (discovery); with probability $1 - \epsilon$, it chooses the action with the highest predicted Q-value (mastery).
- **Decay:** We start with $\epsilon = 1.0$ and slowly decrease it as the agent becomes more proficient.

B. Experience Replay (The Replay Buffer)

In a driving simulation, consecutive frames are nearly identical. If the network learns from these frames in order, it suffers from "catastrophic forgetting" and high data correlation.

- **Mechanism:** The agent stores its experiences (s, a, r, s') in a large memory buffer.
- **Random Sampling:** During training, we pull a random "mini-batch" of memories. This breaks the temporal link between frames and ensures the gradient updates are stable and diverse.

C. Target Networks (The Anchor)

If we use the same network to calculate the prediction and the target, the target moves every time we update the weights. This creates a feedback loop that often leads to divergence.

- **Mechanism:** DQN uses two networks:
    1. **Policy Network (θ):** Updated every step; used to select actions.
    2. **Target Network (θ^-):** A frozen copy used to calculate the stable target. It is synchronized with the Policy Network only every N steps.

3.4 Classification (Taxonomy)

Following the **ABCD** pillars of RL, DQN is classified as:

- **A. Action Space: Discrete**. DQN cannot naturally output a range of numbers. We must discretize the CarRacing controls (e.g., Action 0: Hard Left, Action 1: Straight, Action 2: Hard Right).
- **B. Policy Type: Deterministic**. It seeks the single highest value for a given state.
- **C. Paradigm: Off-Policy**. It reuses data from the Replay Buffer regardless of the current policy.
- **D. Architecture: Value-Based**. It focuses on the Q function rather than a separate Actor.

3.5 Step-by-Step Training Loop

DQN Training Process Schematic

1. **Initialization:** Create the **Policy Network** with random weights.
2. **Initial Sync:** Copy weights from the Policy Network to the **Target Network** to start with identical states.
3. **Navigation:** The agent interacts with the **Environment** using an ϵ-greedy strategy and stores experiences in the **Replay Memory**.
4. **Prediction (Policy):** The Policy Network predicts the current **Q-values** for the sampled state.
5. **Reference (Target):** The Target Network generates **stable reference values** for the next state.
6. **Bellman Equation:** Inputs from the previous steps are processed through the Bellman formula.
7. **Set Target:** Establish the **"Correct Answer"** (Target Vector) based on the Bellman calculation.
8. **Optimization (Update):** The **Loss** box calculates the discrepancy between the target and the current prediction, driving the **Policy Network update** via backpropagation.
9. **Repetition:** The loop (steps 3–8) repeats iteratively to refine the agent's strategy.
10. **Synchronization:** Periodically sync weights from the Policy Network to the Target Network to maintain training stability.

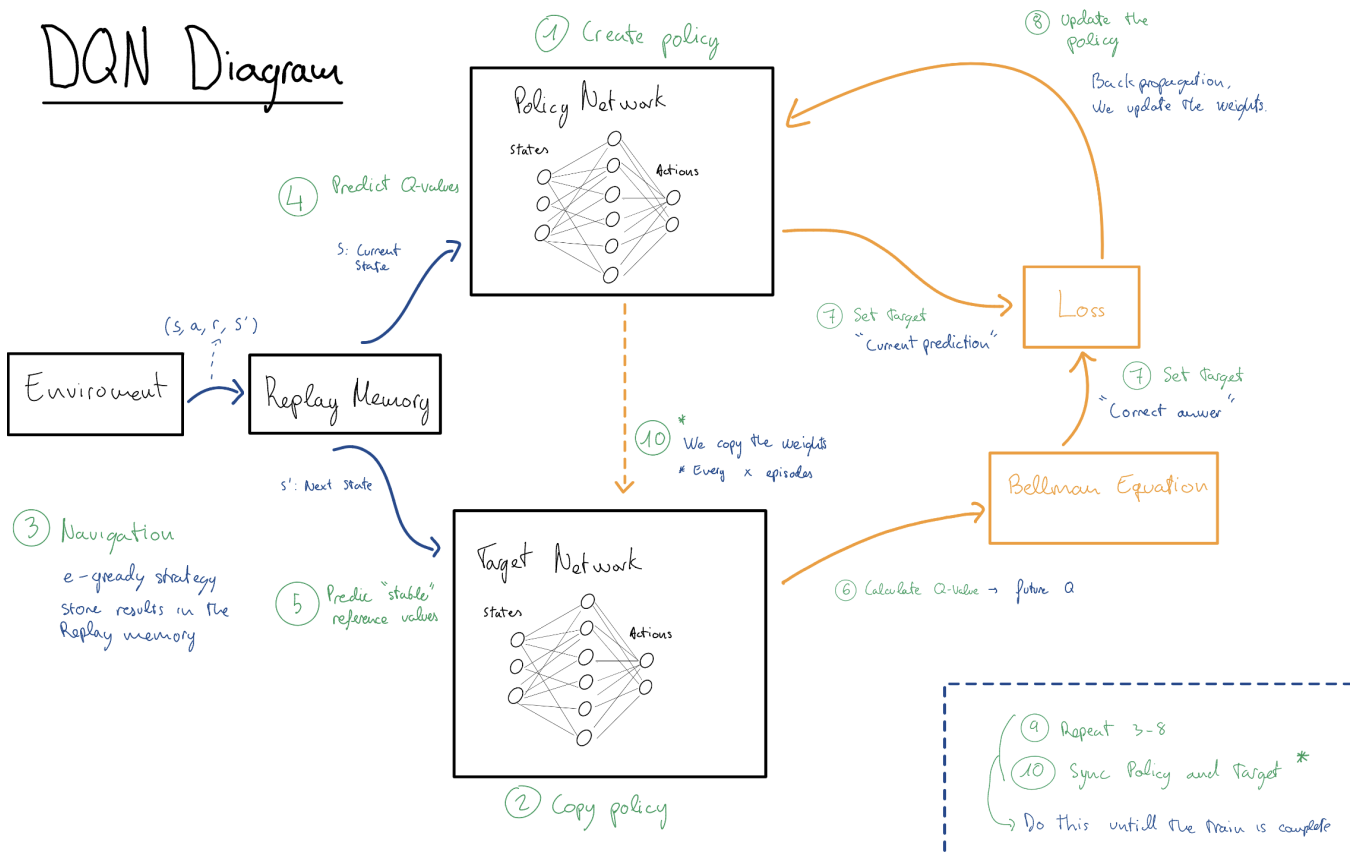

Figure 1: Procedural architecture and data flow of a Deep Q-Network (DQN) training cycle.

---

3. Twin Delayed DDPG (TD3): Mastering Continuity

While DQN excels in discrete spaces, it fails in continuous domains. TD3 was developed to address the **Overestimation Bias** inherent in the Deep Deterministic Policy Gradient (DDPG) algorithm.

3.1 The Overestimation Problem

Value-based methods often overestimate Q-values because they use the maximum of noisy estimates. In continuous control, this leads to the accumulation of errors and suboptimal policies.

3.2 The TD3 "Tricks" for Stability

1. **Clipped Double-Q Learning:** TD3 maintains two independent critics (Q_1, Q_2) and uses the minimum of their estimates to calculate the target:

$$y = r + \gamma \min_{i=1,2} Q_{\theta_{target,i}}(s', \mu_{\phi_{target}}(s') + \epsilon)$$

2. **Target Policy Smoothing:** Small random noise is added to the target actions to smooth out the value function surface, making it less susceptible to exploitation by policy errors.
3. **Delayed Policy Updates:** The Actor network is updated at a lower frequency than the Critic, ensuring that the value function is sufficiently accurate before adjusting the policy.

---

4. Soft Actor-Critic (SAC): Maximum Entropy Framework

SAC is an off-policy actor-critic algorithm that utilizes a stochastic policy. It is distinguished by its focus on **Maximum Entropy Reinforcement Learning**.

4.1 Entropy Regularization

Instead of just maximizing the expected reward, SAC aims to maximize the reward plus the entropy ($\mathcal{H}$) of the policy. Entropy represents the "randomness" or "uncertainty" of the agent's choices. $$J(\pi) = \sum_{t=0}^{T} \mathbb{E}*{(s_t, a_t) \sim \rho*\pi} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))]$$

4.2 Key Advantages

- **Automatic Exploration:** The agent is incentivized to explore all possible actions that lead to high rewards, preventing premature convergence to local optima.
- **Robustness:** Because the policy is stochastic, it is naturally more robust to noise and changes in environment dynamics compared to deterministic methods like TD3.
- **Reparameterization Trick:** To allow backpropagation through stochastic sampling, SAC expresses the action as a deterministic function of the state and independent noise: $a = f_\theta(\epsilon, s)$.

---

5. Comparative Analysis

Feature	DQN	TD3	SAC
Action Space	Discrete	Continuous	Continuous
Policy Nature	Deterministic	Deterministic	Stochastic
Stability Strategy	Target Network	Clipped Double-Q	Entropy Regularization
Exploration	ϵ-greedy	Additive Noise	Entropy Maximization

---

6. Conclusion

The progression from DQN to SAC shows a shift from stabilizing value targets toward stabilizing the exploration process itself. DQN introduced the necessary infrastructure (Buffers and Targets), TD3 refined the value estimation through pessimism (Clipped Q), and SAC integrated exploration directly into the mathematical objective through Entropy. Understanding these differences allows researchers to select the most appropriate tool based on the complexity and continuity of the task at hand.

4. The Evolutionary Journey: Why we moved forward

1. DQN to DDPG: Crossing the Continuous Gap

DQN was limited to buttons. **DDPG** introduced the Actor-Critic framework to off-policy learning, allowing for "pedal" control. However, DDPG was prone to "divergence"—it would often learn a bad policy and never recover because its Critic was too optimistic.

2. DDPG to TD3: Implementing "Pessimism"

TD3 fixed DDPG by adding three stability mechanisms:

- **Clipped Double-Q:** Using two critics and taking the minimum prevents the agent from overestimating how good a state is.
- **Target Policy Smoothing:** Adding noise to targets prevents the actor from exploiting sharp "peaks" in the value function.
- **Update Rule:**

$$y = r + \gamma \min_{i=1,2} Q_{\theta_{target,i}}(s', \mu_{\phi_{target}}(s') + \epsilon)$$

3. TD3 to SAC: Embracing Randomness (Entropy)

While TD3 is precise, it can be rigid. **SAC** introduced **Entropy Regularization**. Instead of just trying to get a high score, SAC tries to get a high score while being as "random" as possible.

- **Entropy ($\mathcal{H}$):** This ensures the agent never stops exploring until it is absolutely certain of the optimal path.
- **Objective:**

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))]$$

---

5. Summary for Students

- If your task is **discrete** (like Chess): Use **DQN**.
- If you need **simple, reliable** continuous control and have time to train: Use **PPO**.
- If you need **maximum efficiency** and robust exploration in continuous space: Use **SAC**.
- If you need **deterministic precision** (robotics): Use **TD3**.